Chiffrer des données : *de PKCS#1 v1.5 à HPKE*

Symétrique, asymétrique, hybride et post-quantique : l'évolution des schémas et le vocabulaire du cours

F1 · RECTO · symétrique

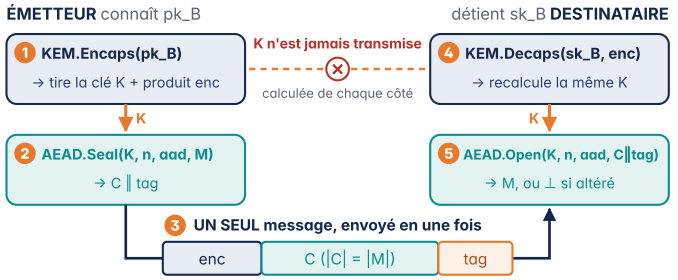
Introduction à la cryptographie moderne
Benoit LEGER-DERVILLE · Stelau · iDAKTO

1 L'idée-force : tout chiffrement réel est hybride

L'asymétrique est lent et ne chiffre que quelques dizaines d'octets ; le symétrique est rapide mais exige une clé partagée. On combine donc les deux : **l'asymétrique transmet (ou fait naître) une clé de session *K***, puis **un chiffrement symétrique authentifié (AEAD, voir p. 4) protège les données sous *K***.

La théorie moderne nomme ces deux étages **KEM** (*Key Encapsulation Mechanism*) et **DEM** (*Data Encapsulation Mechanism*) — Cramer & Shoup 2003, ISO/IEC 18033-2 (2006). KEM IND-CCA2 + DEM sûr ⇒ ensemble IND-CCA2 : on **raisonne et remplace chaque étage séparément** (crypto-agilité, migration post-quantique).

Attention au mot « hybride », qui a deux sens dans le cours : (1) *chiffrement hybride* = asymétrique + symétrique (KEM/DEM) ; (2) *KEM hybride post-quantique* = classique + post-quantique combinés (ex. X25519 + ML-KEM-768), sûr si l'un des deux résiste.



Le destinataire découpe : enc → étape 4, C ‖ tag → étape 5. Le nonce n est dérivé avec K ou joint en clair.

|enc| : 65 o (ECIES P-256) · 32 o (X25519) · 256 o (RSA-2048) · 1 088 o (ML-KEM-768) · tag : 16 o

Ancienne école : « enveloppe digitale » = RSA chiffre une clé AES choisie par l'émetteur.

2 Le symétrique en 25 ans : du chiffrement par blocs à l'AEAD

AES n'est qu'une permutation de 16 octets

AES (Rijndael, 2001) chiffre **un bloc de 128 bits** sous une clé de 128, 192 ou 256 bits : c'est une **permutation pseudo-aléatoire**, c'est-à-dire une bijection sur les 2^{128} blocs possibles (pas un réarrangement des octets : S-box, ShiftRows, MixColumns et ajout de clé sont tous inversibles). Le **mode opératoire** fixe l'enchaînement des blocs, l'IV/nonce, le remplissage et, depuis 2004, l'**intégrité**.

Bascule : de « chiffrement seul, MAC à part » (souvent mal composés) à l'**AEAD** (p. 4), qui chiffre, authentifie et **rejette** tout chiffré altéré.

1977 DES (FIPS 46) : bloc 64 bits, clé 56 bits ; cassé par force brute en 1998.

1981 Modes **ECB, CBC, CFB, OFB** (FIPS 81) ; **3DES** prolonge DES (retiré fin 2023).

2001 AES (FIPS 197) ; mode **CTR** normalisé (SP 800-38A).

2002 Oracle de remplissage sur CBC (Vaudenay) → Lucky13, POODLE. **AES-KW** (RFC 3394).

2004-07 CCM puis **GCM** (SP 800-38D) : premiers AEAD normalisés. XTS (IEEE 1619).

2015 ChaCha20-Poly1305 (RFC 7539 → 8439), AEAD sans AES matériel.

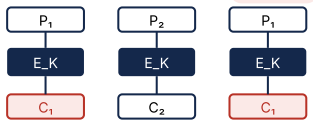
2018 TLS 1.3 (RFC 8446) : AEAD uniquement, CBC et RC4 supprimés.

2019 AES-GCM-SIV (RFC 8452) : tolère la réutilisation de nonce.

3 Les modes d'AES en un coup d'œil

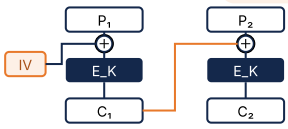
E_K(x) = bloc x chiffré par AES sous la clé K (**D_K** : l'inverse) · CTR et GCM n'utilisent que E_K

ECB — à bannir



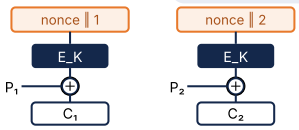
Chaque bloc chiffré indépendamment : **blocs clairs égaux ⇒ blocs chiffrés égaux** (le « pingouin » reste visible). Déterministe, aucune sémantique de sécurité.

CBC — historique



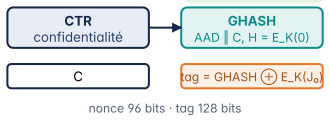
Chaînage par XOR avec le chiffré précédent ; IV **imprévisible** ; remplissage PKCS#7. Séquentiel au chiffrement. Sans MAC : **oracle de remplissage**.

CTR — brique de base



AES devient un **générateur de flux** : C = P ⊕ E_K(compteur). Parallélisable, pas de remplissage. **Malléable** (bit flip) et **nonce unique impératif**.

GCM — AEAD standard



nonce 96 bits · tag 128 bits

CTR + MAC polynomial dans GF(2^{128}). Très rapide (AES-NI, PCLMUL). **Réutiliser un nonce** révèle $P_1 \oplus P_2$ et la clé de hachage H ⇒ falsifications.

Mode	Type	IV / nonce	Intégrité	Point d'attention	Usage typique	Statut
CONFIDENTIALITÉ SEULE (À COMPOSER AVEC UN MAC : ENCRYPT-THEN-MAC)						
ECB	bloc	aucun	non	Fuit la structure du clair ; déterministe.	Jamais (sauf 1 bloc aléatoire)	proscrit
CBC	bloc chaîné	16 o imprévisible	non	Oracle de remplissage ; IV prévisible ⇒ BEAST (TLS 1.0).	TLS ≤ 1.2, CMS ancien, fichiers	legacy
CFB / OFB	flux	16 o unique	non	Variantes historiques de chiffrement en flux.	Protocoles anciens, OpenPGP (CFB)	legacy
CTR	flux	nonce ‖ compteur unique	non	Réutilisation ⇒ $P_1 \oplus P_2$; malléable.	Brique de GCM/CCM, SRTP	brique
XTS	bloc ajustable	n° de secteur	non	Conçu pour le disque (pas d'expansion) ; pas d'authentification.	FileVault, BitLocker, LUKS	spécialisé
AEAD : CHIFFREMENT AUTHENTIFIÉ AVEC DONNÉES ASSOCIÉES						
GCM	CTR + GHASH	96 bits, **jamais réutilisé**	tag 128	Nonce aléatoire : ≤ 2^{32} messages par clé. Tags courts déconseillés.	TLS 1.2/1.3, IPsec, JWE, CryptoKit, HPKE	référence
CCM	CTR + CBC-MAC	7-13 o unique	tag 4-16	Deux passes, non « en ligne » ; adapté au matériel contraint.	Wi-Fi WPA2, Bluetooth LE, IoT	OK
ChaCha20-Poly1305	flux + MAC	96 bits unique	tag 128	Pas d'AES : rapide en logiciel, constant-time par construction.	TLS 1.3, WireGuard, SSH, HPKE	référence
AES-GCM-SIV	SIV	96 bits (tolérant)	tag 128	Réutiliser un nonce ne révèle que l'égalité de messages.	Nonces difficiles à garantir, cloud KMS	robuste
AES-KW / KWP	key wrap	aucun (déterministe)	intégré	Réservé au chiffrement de **clés** (RFC 3394 / 5649, SP 800-38F).	HSM, CMS, JWE A256KW, PKCS#11	clés

Règles d'or du symétrique

- **Toujours un AEAD** ; si ce n'est pas possible : *encrypt-then-MAC*, clés distinctes, comparaison du MAC en temps constant.
- **Un nonce ne sert qu'une fois par clé** (CTR, GCM, ChaCha20) : compteur, ou aléa 96 bits avec rotation de clé.
- **Ne jamais révéler pourquoi** un déchiffrement échoue (remplissage, MAC, format) : un message d'erreur distinct est un oracle.

Notions de sécurité à retenir

- **IND-CPA** : le chiffré ne révèle rien du clair face à qui peut chiffrer. Exige un chiffrement *probabiliste* (IV/nonce).
- **IND-CCA2** : idem même si l'attaquant fait déchiffrer d'autres chiffrés. C'est le niveau exigé aujourd'hui ; l'AEAD l'apporte.
- **INT-CTXT** : impossible de forger un chiffré accepté. IND-CPA + INT-CTXT ⇒ IND-CCA2.

Références : FIPS 197 · SP 800-38A/C/D/F · RFC 3394, 5649, 8439, 8452, 8446 · ISO/IEC 18033-2

Transmettre une clé : *du transport au KEM*

RSA, courbes elliptiques, HPKE et post-quantique : pourquoi PKCS#1 v1.5 est sorti et ce qui l'a remplacé

Introduction à la cryptographie moderne
Benoit LEGER-DERVILLE · Stelau · iDAKTO

4 Cinquante ans d'histoire en trois couloirs

	1976 – 1997 Fondations	1998 – 2005 Attaques et preuves	2006 – 2017 Normalisation	2018 – 2023 Rupture	2024 – 2035 Post-quantique
RSA	**1977** RSA (Rivest, Shamir, Adleman) **1993** PKCS#1 v1.5 : remplissage `00 02 PS 00 M` **1994** OAEP (Bellare, Rogaway)	**1998** Bleichenbacher (« million de messages ») **1998** PKCS#1 v2.0 adopte OAEP **2001** Manger : attaque d'OAEP mal implémenté **2001** RSA-KEM (Shoup)	**2006** ISO/IEC 18033-2 : RSA-KEM **2010** RSA-KEM dans CMS (RFC 5990) **2016** PKCS#1 v2.2 (RFC 8017) **2017** ROBOT : Bleichenbacher toujours actif en TLS	**2018 TLS 1.3** supprime le transport de clé RSA **2019** SP 800-131A r2 : chiffrement v1.5 de clés **interdit après 2023** **2023** Marvin : variante temporelle, nombreuses bibliothèques	**2025** RSA-KEM dans CMS révisé (RFC 9690) **2030** RSA 112 bits déprécié **2035** RSA interdit (NIST IR 8547, projet)
DH / courbes	**1976** Diffie-Hellman, « New Directions » **1985** ElGamal ; courbes elliptiques (Koblitz, Miller)	**1998** DHAES / DHIES (Abdalla, Bellare, Rogaway) **2000-04 ECIES** : SEC 1, ANSI X9.63, IEEE 1363a	**2006** Curve25519 (Bernstein) **2010** HKDF (RFC 5869) **2015** JWE ECDH-ES (RFC 7518) **2016** X25519 (RFC 7748)	**2022 HPKE** (RFC 9180) **2023** MLS (RFC 9420) ; HPKE dans CryptoKit TLS ECH, Oblivious HTTP	**2030 / 2035** même calendrier de retrait que RSA pour ECDH / ECDSA ECC conservé dans les **hybrides**
Post-quantique	**1994 Shor** : factorisation et log discret quantiques	**1996** NTRU, premier chiffrement sur réseaux euclidiens pratique	**2016** appel à candidatures NIST PQC	**2022** Kyber, Dilithium, SPHINCS+ retenus **2023** premiers tests TLS hybrides (X25519Kyber768)	**2024 FIPS 203/204/205** : ML-KEM, ML-DSA, SLH-DSA **2024-25** TLS hybride par défaut (navigateurs) **2025** CryptoKit et Secure Enclave

5 Trois manières d'obtenir la clé de session

Transport de clé

L'émetteur **choisit** K et la **chiffre** sous pk.

RSAES-PKCS1-v1_5 interdit pour les clés

```
EM = 00 ‖ 02 ‖ PS ‖ 00 ‖ K
EM : message encodé · PS : bourrage ≠ 00
c = EMᵉ mod n
```

Le destinataire qui signale (même indirectement : erreur, délai) si « 00 02 » est présent offre un **oracle** : ~10^6 requêtes suffisent à déchiffrer c (Bleichenbacher 1998). Les contre-mesures en temps constant échouent régulièrement (ROBOT 2017, Marvin 2023).

RSAES-OAEP OK

```
DB = lHash ‖ PS ‖ 01 ‖ K
maskedDB = DB ⊕ MGF1(seed)
maskedSeed = seed ⊕ MGF1(maskedDB)
EM = 00 ‖ maskedSeed ‖ maskedDB
```

Bellare & Rogaway 1994 : remplissage randomisé par un réseau de Feistel à deux tours, **IND-CCA2** dans le modèle de l'oracle aléatoire. Reste sensible aux fuites d'implémentation (Manger 2001).

Accord de clé (Diffie-Hellman)

Les deux parties **contribuent** ; K est dérivée d'un secret commun.

ECDH éphémère-statique (« ECDH-ES »)

```
E = e·G (e éphémère)
Z = x(e·Q_B) = x(d_B·E)
K = KDF(Z, info)
```

Le secret brut Z n'est **jamais** utilisé directement : il passe par un **KDF** (X9.63-KDF, HKDF = *extract* puis *expand*, RFC 5869) qui produit une clé uniforme et la lie au contexte.

ECIES = ECDH-ES + KDF + DEM
OK, peu interopérable

```
envoi : E ‖ AEAD(K, M)
Apple : X9.63-SHA256, sharedInfo = E,
KⅡV sur 32 o, AES-128-GCM
```

DHAES/DHIES (Abdalla, Bellare, Rogaway 1998), SEC 1, X9.63, IEEE 1363a, ISO 18033-2 : **une famille** plutôt qu'un schéma (KDF, sharedInfo, encodage du point, DEM, cofacteur varient).

Encapsulation de clé (KEM)

K est **générée par l'encapsulation** elle-même : nul ne la « choisit ».

```
(K, enc) ← Encaps(pk) · K ← Decaps(sk, enc)
```

RSA-KEM (Shoup 2001) OK, rare : r aléatoire dans [0, n), enc = r^e mod n, K = KDF(r). Aucun remplissage, preuve IND-CCA2 serrée.

HPKE — RFC 9180 (2022) standard moderne : bâti sur **DHKEM** (ECDH-ES vu comme un KEM).

```
suite = KEM + KDF + AEAD (X25519, HKDF-SHA256, AES-128-GCM)
KeySchedule(mode, ss, info, psk) → key, base_nonce, exporter
```

Modes **Base** (≈ ECIES propre), **PSK**, **Auth** (émetteur authentifié), **AuthPSK**.

ML-KEM (FIPS 203) PQ : Module-LWE + transformation FO ; KEM natif, insérable dans HPKE.

6 Comparatif des schémas de chiffrement à clé publique

Schéma	Paradigme	Fondement / sécurité	Surcoût transmis	Où on le trouve	Statut 2026
RSAES-PKCS1-v1_5	transport	Aucune preuve ; oracle	|n| (256 o en 2048)	TLS ≤ 1.2 « RSA kx », vieux CMS / S/MIME	interdit (clés)
RSAES-OAEP	transport	RSA, IND-CCA2 (ROM)	|n| (256 o)	CMS, JWE `RSA-OAEP-256`, PKCS#11, Apple `OAEPSHA256AESGCM`	OK en sursis
RSA-KEM	KEM	RSA, IND-CCA2 serrée (ROM)	|n| (256 o)	CMS (RFC 9690)	OK en sursis
ECIES (P-256)	KEM + DEM	gap-DH, IND-CCA2 (ROM)	65 o + tag 16 o	Apple `SecKey` (Secure Enclave), cartes, eID	OK en sursis
JWE ECDH-ES	accord + KDF	ECDH, Concat-KDF	epk (≈ 65 o)	JOSE, wallets EUDI, OID4VP (réponses chiffrées)	OK en sursis
HPKE (X25519)	KEM/DEM unifié	Analyse formelle, IND-CCA2	32 o + tag 16 o	MLS, ECH, OHTTP, CryptoKit (macOS 14+)	référence
ML-KEM-768	KEM	Module-LWE, IND-CCA2, PQ	enc 1 088 o	FIPS 203, CryptoKit + Secure Enclave (macOS 26)	post-quantique
X25519MLKEM768 / X-Wing	KEM hybride	Sûr si X25519 ou ML-KEM tient	1 120 o	TLS 1.3 (navigateurs), HPKE (draft X-Wing)	transition

7 Glossaire du cours

AEAD / AAD — chiffrement authentifié ; données associées authentifiées mais non chiffrées (en-têtes).

IV / nonce — valeur publique par message. IV : souvent imprévisible (CBC) ; nonce : seulement *unique* (CTR, GCM).

Oracle — tout comportement observable (erreur, délai) qui répond à une question sur un chiffré ; base des attaques Vaudenay et Bleichenbacher.

Malléabilité — modifier un chiffré pour transformer le clair de façon contrôlée (CTR, RSA sans remplissage).

Transport vs accord de clé — K choisie et chiffrée par un seul vs K dérivée des contributions des deux.

KEM / DEM — encapsulation de la clé / encapsulation des données ; composition sûre et interchangeable.

KDF — dérivation de clé : X9.63, Concat-KDF (SP 800-56C), HKDF (extract → expand).

Confidentialité persistante (*forward secrecy*) — la compromission d'une clé long terme ne révèle pas les sessions passées ; absente du transport RSA et d'ECIES (clé statique).

Cofacteur — rapport entre l'ordre de la courbe et celui du sous-groupe (1 pour P-256, 8 pour Curve25519).

ROM / réduction serrée — hachage idéalisé dans la preuve ; perte de sécurité faible entre schéma et problème (p. 4).

Transformation FO — Fujisaki-Okamoto : rend un KEM IND-CCA2 en re-chiffrant pour vérifier (ML-KEM).

Rejet implicite — un chiffré invalide donne une clé pseudo-aléatoire plutôt qu'une erreur : pas d'oracle (ML-KEM).

HNDL (*harvest now, decrypt later*) — capter aujourd'hui pour déchiffrer avec un ordinateur quantique : pourquoi le chiffrement migre avant la signature.

Crypto-agilité — pouvoir changer d'algorithme sans refondre le protocole (suites HPKE, OID, identifiants JOSE).

Ne pas confondre les deux « v1.5 » : le *chiffrement* RSAES-PKCS1-v1_5 est proscrit pour transporter des clés (interdit après 2023, absent de TLS 1.3) ; la *signature* RSASSA-PKCS1-v1_5 reste acceptée (aucun oracle de déchiffrement), RSASSA-PSS lui étant préférée.

Signer : *de PKCS#1 v1.5 à SLH-DSA*

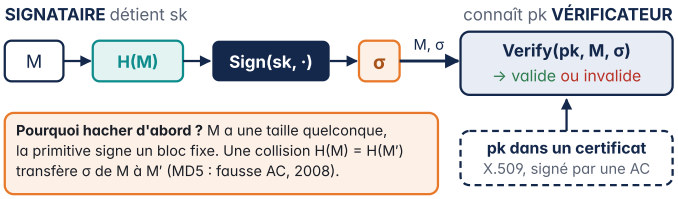
RSA, courbes elliptiques, réseaux euclidiens et fonctions de hachage : l'évolution des schémas de signature

Introduction à la cryptographie moderne
Benoit LEGER-DERVILLE · Stelau · iDAKTO

8 Signature et MAC : ce qui change

Produite avec la **clé privée**, une signature est vérifiable par **quiconque détient la clé publique** (diffusée dans un certificat X.509). On signe le **haché** H(M) : le hachage résume, la clé privée prouve l'origine.

	MAC (HMAC, GHASH…)	Signature
Clé	secret partagé K	biclé (sk, pk)
Qui vérifie	les détenteurs de K	tout le monde
Qui peut produire	tout vérificateur aussi	le seul détenteur de sk
Non-répudiation	non	oui : base de la signature électronique (eIDAS)
Sécurité	**EUF-CMA** (p. 4) : même après des signatures de messages choisis, impossible de signer un message *nouveau*. **SUF-CMA** : ni de re-signer autrement un message déjà signé.	



Pourquoi hacher d'abord ? M a une taille quelconque, la primitive signe un bloc fixe. Une collision H(M) = H(M') transfère σ de M à M' (MD5 : fausse AC, 2008).

Signer ≠ « chiffrer avec la clé privée » : vrai seulement pour RSA sans remplissage ; faux pour ECDSA, EdDSA, ML-DSA, SLH-DSA.

9 Quarante-cinq ans de signatures en trois couloirs

	1976 – 1997 Fondations	1998 – 2009 Normalisation	2010 – 2017 Accidents et renouveau	2018 – 2023 Consolidation	2024 – 2035 Post-quantique
RSA	**1977** RSA **1993 PKCS#1 v1.5** **1996** PSS (Bellare, Rogaway)	**2002** PKCS#1 v2.1 : **RSASSA-PSS** **2006 Bleichenbacher** : faux certificats (e = 3, vérification laxiste)	**2014 BERserk** : même faille dans NSS (Firefox) **2016** PKCS#1 v2.2 (RFC 8017) **2017** SHAttered : collision SHA-1	**2018 TLS 1.3** : PSS pour la poignée de main, v1.5 toléré dans les certificats	**2030** RSA 112 bits déprécié **2035** RSA interdit (NIST IR 8547, projet)
Log discret / courbes	**1985** ElGamal **1989** Schnorr **1992** ECDSA (Vanstone) **1994** DSA (FIPS 186)	**1999-2000** ECDSA normalisé (ANSI X9.62, FIPS 186-2)	**2010 PS3** : k fixe, clé privée retrouvée **2011** Ed25519 · **2017** RFC 8032 **2013** ECDSA déterministe (RFC 6979)	**2019 Minerva**, TPM-Fail : fuites temporelles sur k **2023** FIPS 186-5 : EdDSA approuvé, DSA retiré	**2030 / 2035** même calendrier que RSA ECDSA omniprésent : passkeys, mdoc, wallets EUDI
Hachage et réseaux	**1979** Lamport : signature à usage unique **1979** arbres de Merkle	**2006** GGH, NTRUSign (réseaux) cassés (Nguyen, Regev)	**2011** XMSS **2015** SPHINCS **2017** Dilithium, Falcon, SPHINCS+ soumis au NIST	**2018-19** XMSS (RFC 8391), LMS (RFC 8554) **2020** SP 800-208 **2022** Dilithium, Falcon, SPHINCS+ retenus	**2024 ML-DSA** (FIPS 204), **SLH-DSA** (FIPS 205, ex-SPHINCS+) FN-DSA (Falcon) : FIPS 206 à venir

10 Trois familles, leurs équations et leurs pièges

RSA

RSASSA-PKCS1-v1_5 accepté, legacy

```
EM = 00 ‖ 01 ‖ FF…FF ‖ 00 ‖ DigestInfo
σ = EM^d mod n · déterministe
```

Aucun oracle de déchiffrement ici. Le danger est un **vérificateur laxiste** qui n'analyse pas tout EM (e = 3 : falsifications 2006, 2014).

RSASSA-PSS recommandé

```
H' = H(0⁸ ‖ H(M) ‖ sel)
EM = (PS ‖ 01 ‖ sel) ⊕ MGF1(H') ‖ H' ‖ BC
```

Sel aléatoire, donc signature probabiliste ; preuve **serrée** dans la ROM. Signature = taille de n.

Courbes elliptiques

ECDSA OK

```
k aléa secret, unique par signature
r = x(k·G) mod q
s = k⁻¹(H(M) + d·r) mod q · σ = (r, s)
```

Tout repose sur k : réutilisé ⇒ deux équations, deux inconnues ⇒ d retrouvée (PS3) ; quelques bits biaisés suffisent (Minerva). Parade : **k déterministe** (RFC 6979). (r, −s) est aussi valide : ECDSA n'est pas SUF-CMA.

EdDSA (Ed25519) OK

```
r = H(préfixe ‖ M) · R = r·B
S = r + H(R ‖ A ‖ M)·a · σ = (R, S)
```

Schnorr sur Curve25519 : nonce déterministe par construction, temps constant.

Post-quantique

ML-DSA (FIPS 204, ex-Dilithium) réseaux

```
y aléatoire · w = A·y · c = H(μ ‖ w₁)
z = y + c·s₁ · rejet et nouvel essai
si z révèle s₁ (Fiat-Shamir with aborts)
```

Module-LWE / SIS. Rapide ; « hedged » par défaut (aléa + déterminisme).

SLH-DSA (FIPS 205, ex-SPHINCS+) hachage

Hyper-arbre de Merkle (WOTS+, FORS). Sécurité fondée **uniquement** sur le hachage : le choix conservateur. Sans état, mais signatures de 8 à 50 Ko, lentes.

LMS / XMSS hachage, avec état

Plus compacts, mais chaque feuille ne sert **qu'une fois** : état restauré ou cloné ⇒ falsifications. Firmware (CNSA 2.0).

11 Comparatif des schémas de signature

Schéma	Fondement	pk	signature	Déterministe	Où on le trouve	Statut 2026
RSASSA-PKCS1-v1_5	RSA-2048	256 o	256 o	oui	Certificats X.509, CMS, CAdES/PAdES, RS256	legacy sursis
RSASSA-PSS	RSA-2048	256 o	256 o	non (sel)	TLS 1.3, JWS PS256, signature qualifiée	OK sursis
ECDSA P-256	log discret EC	64 o	64 o	option RFC 6979	Secure Enclave, passkeys, mdoc, wallets EUDI, ES256	OK sursis
Ed25519	Schnorr / EC	32 o	64 o	oui	SSH, Signal, DNSSEC, FIPS 186-5	OK sursis
ML-DSA-65	Module-LWE / SIS	1 952 o	3 309 o	hedged	CryptoKit + Secure Enclave ; X.509 et TLS en cours	post-quantique
FN-DSA-512 (Falcon)	réseaux NTRU	897 o	≈ 666 o	non	Contraintes de taille ; flottants délicats à protéger	en cours
SLH-DSA-128s	hachage seul	32 o	7 856 o	option	Racines de confiance, archivage long terme	conservateur
LMS / XMSS	hachage, avec état	≈ 60 o	≈ 1-3 Ko	oui	Firmware, démarrage sécurisé, HSM gérant l'état	niche

Pièges classiques

- **Nonce DL/EC** : unique, uniforme et secret, ou déterministe (RFC 6979, EdDSA).
- **Vérifier strictement** : réencoder l'EM attendu et comparer, plutôt que l'analyser (RSA v1.5).
- **Hachages cassés** : MD5 et SHA-1 interdits ; LMS/XMSS : jamais d'état cloné ou restauré.

Signature et transition post-quantique

- **Pas de HNDL** : forger exige l'ordinateur quantique *au moment* de la falsification ; le chiffrement migre donc en premier.
- **Sauf ce qui dure** (AC racines, firmware, archives) ; signatures **composites** classique + PQ en cours à l'IETF.

Références : RFC 8017 · FIPS 186-5 · RFC 6979 · RFC 8032 · FIPS 204 · FIPS 205 · RFC 8391, 8554 · SP 800-208 · NIST IR 8547 (projet)

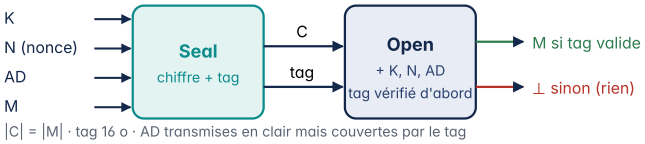
Comprendre : *les notions derrière les planches*

AEAD, MAC et hachage, notations, jeux de sécurité, preuves et vocabulaire PKCS#1

Introduction à la cryptographie moderne
Benoit LEGER-DERVILLE · Stelau · iDAKTO

12 L'AEAD : chiffrer et authentifier en une seule primitive

AEAD = *Authenticated Encryption with Associated Data*. Primitive symétrique unique qui remplace le couple « chiffrement + MAC ».



|C| = |M| · tag 16 o · AD transmises en clair mais couvertes par le tag

- **Apporte** confidentialité de M et authenticité de AD et C ; **Open vérifie avant de rendre** : un bit altéré donne ⊥, jamais un clair partiel.
- **AD** (*associated data*) : en-tête de paquet, numéro de séquence, identifiant de clé. Lisibles par le réseau, mais non modifiables.
- **Pourquoi une primitive à part** : composer soi-même est piégeux. TLS ≤ 1.2 faisait *MAC-then-encrypt* ⇒ oracles de remplissage (Lucky13, POODLE). Seul *encrypt-then-MAC* est sûr de façon générique (Bellare, Namprempre 2000). L'AEAD enferme une composition prouvée dans une seule boîte : AES-GCM, ChaCha20-Poly1305.

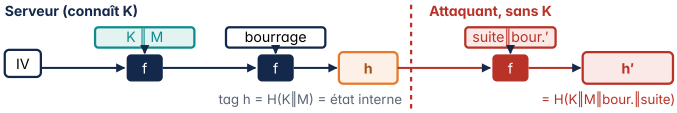
Le MAC interne d'un AEAD n'est presque jamais SHA, mais un **hachage universel** polynomial (voir 13 pour la différence MAC / hachage) :

AEAD	MAC interne	Principe
AES-GCM	GHASH	polynôme en H = E_K(0) dans GF(2^{128}), masqué par E_K(J_0)
ChaCha20-Poly1305	Poly1305	polynôme en r modulo $2^{130} - 5$, masqué par s
AES-CCM	CBC-MAC	AES en CBC, on garde le dernier bloc
A128CBC-HS256	HMAC-SHA-256	CBC + HMAC en encrypt-then-MAC (JWE)

13 Un MAC n'est pas un hachage

	Hachage (SHA-256)	MAC
Clé	aucune	secrète : tag = MAC(K, M)
Qui calcule	tout le monde	les seuls détenteurs de K
Protège contre	l'erreur accidentelle	la modification délibérée
Propriétés	sens unique, collisions	infalsifiable sans K

Pourquoi pas simplement `H(K ‖ M)` ? L'extension de longueur. SHA-256 (construction de Merkle-Damgård) découpe l'entrée en blocs de 64 o et enchaîne une compression f sur un état de 256 bits ; le haché publié **est** l'état final, et le bourrage (0x80, zéros, longueur) ne dépend que de la longueur.

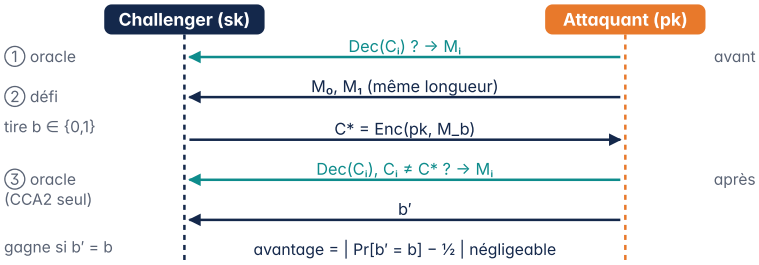


L'attaquant qui voit h et devine |K| **reprend le calcul à partir de h** et obtient un tag valide pour le message prolongé `M ‖ bourrage ‖ suite`, sans connaître K (API Flickr, 2009). Sont immunisés : **HMAC** (RFC 2104), `H((K⊕opad) ‖ H((K⊕ipad) ‖ M))`, dont le hachage externe masque l'état ; **SHA-3 / KMAC** (éponge dont une partie de l'état n'est jamais publiée) ; les versions tronquées (SHA-512/256).

> **Wegman-Carter (1981)** : hachage universel à clé secrète + masque à usage unique ⇒ falsification avec une probabilité ≈ ℓ / 2^{128} (ℓ : longueur en blocs), **sans hypothèse** sur une fonction de hachage. Très rapide (PCLMUL, PMULL). **Revers** : seul, un hachage universel n'est ni à sens unique ni résistant aux collisions. **Nonce réutilisé** ⇒ même masque ⇒ $\text{tag}_1 \oplus \text{tag}_2$ = polynôme dont H est racine ⇒ H retrouvée ⇒ falsifications à volonté.

14 Les jeux de sécurité : IND-CCA2 et ses cousins

Une notion de sécurité est un **jeu** entre un challenger et un attaquant. Le schéma est sûr si aucun attaquant réaliste n'y gagne nettement mieux qu'au hasard.



Notion	Ce que l'attaquant peut faire
IND-CPA	chiffrer seulement (avec pk, il le peut toujours) ; exige un chiffrement **probabiliste**
IND-CCA1	+ oracle de déchiffrement **avant** le défi (①)
IND-CCA2	+ oracle **avant et après**, sauf sur C* (① et ③) : le niveau exigé aujourd'hui
EUF-CMA	signatures et MAC : obtient des signatures de messages choisis, doit signer un message nouveau

Un schéma **malléable** n'est jamais IND-CCA2 : on transforme C* en C', on fait déchiffrer C' et on en déduit M_b. **Bleichenbacher, c'est exactement ce jeu** : le serveur TLS est l'oracle (« remplissage valide ou non »), et RSA est malléable (c·s^e chiffre m·s).

15 Preuves : ROM et réduction serrée

Réduction : « si un attaquant A casse le schéma avec une probabilité ε en temps t, on construit B qui résout RSA (ou DH, LWE…) avec une probabilité ε' en temps t' ». Le schéma est alors au moins aussi dur que le problème… à la perte près.

Serrée (*tight*) : ε' ≈ ε et t' ≈ t. Casser le schéma est aussi dur que le problème : RSA-KEM, PSS. **Lâche** : par exemple ε' ≈ ε / q, avec q ≈ 2^{60} requêtes de hachage. La preuve reste vraie mais ne garantit presque rien aux tailles usuelles : OAEP (Fujisaki, Okamoto, Pointcheval, Stern 2001).

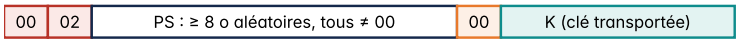
ROM (*Random Oracle Model*, Bellare et Rogaway 1993) : dans la preuve, la fonction de hachage est remplacée par une fonction **vraiment aléatoire**, consultable comme une boîte noire. C'est une idéalisation : SHA-256 n'est pas aléatoire, et il existe des contre-exemples (artificiels). Une preuve sans cette hypothèse est dite **dans le modèle standard** (ex. Cramer-Shoup 1998).

16 Notations des planches

`K`	clé symétrique (128-256 bits)	`(pk, sk)`	biclé publique / privée
`E_K, D_K`	AES sous K, et son inverse	`‖`	concaténation
⊕	XOR bit à bit	`⊥`	rejet, erreur
`G, B`	point générateur de la courbe	`d·G`	multiplication scalaire
`x(P)`	abscisse du point P	`q`	ordre du sous-groupe
`n, e, d`	module et exposants RSA	`H`	fonction de hachage
`σ`	signature	`enc`	encapsulation transmise

17 Le vocabulaire de PKCS#1 et l'attaque de Bleichenbacher

Chiffrement RSAES-PKCS1-v1_5 (block type 2)



Signature RSASSA-PKCS1-v1_5 (block type 1)



EM = message encodé, de la taille de n (256 o en RSA-2048), puis élevé à la puissance e ou d.

`RSAES / RSASSA`	RSA Encryption Scheme / RSA Signature Scheme with Appendix
`EM · PS`	Encoded Message · Padding String (le 00 qui suit PS marque le début de K, d'où PS ≠ 00)
`lHash · MGF1`	(OAEP) haché d'un label optionnel · fonction de masquage bâtie sur un haché
`sel · BC`	(PSS) aléa qui randomise la signature · octet final 0xBC

Mécanisme de l'attaque (1998). Soit k la taille de n en octets et B = 2^(8(k−2)). Un EM valide commence par `00 02`, donc le clair m appartient à **[2B, 3B[**.

- RSA est **multiplicatif** : c' = c · s^e mod n est le chiffré de m · s mod n, calculable sans clé.
- L'attaquant soumet c' pour des s bien choisis. Chaque réponse « remplissage valide », par un message d'erreur, une alerte TLS ou un simple **délai**, lui apprend que m·s mod n ∈ [2B, 3B[.
- Chaque réponse positive **resserre l'intervalle** qui contient m. Après 10^4 à 10^6 requêtes, il est réduit à un point : m, donc K, est retrouvé **sans jamais connaître d**.
- Les parades « ne rien laisser paraître » (TLS 1.2 : clé aléatoire de substitution) sont difficiles à tenir en temps constant : ROBOT (2017), Marvin (2023). D'où l'abandon du transport v1.5 au profit d'OAEP, puis des KEM.

Références : RFC 5116 (AEAD) · RFC 2104 (HMAC) · Wegman-Carter 1981 · Bellare-Rogaway 1993 · Bellare-Namprempre 2000 · Bleichenbacher, CRYPTO 1998 · RFC 8017